

Королев Адам Маратович

НФИмд-01-25

Лабораторная работа №2

1. Столбцовая перестановка с ключевым словом (маршрутное шифрование по столбцам).
2. Шифрование решеткой Флейснера.
3. Шифр Виженера по русскому алфавиту без буквы "ё".

Базовые функции

Ниже защиты вспомогательные функции для работы с русским текстом. Алфавит:
"абвгдежзийклмнопрстуфхцшщъыьэюя". Буква "ё" при нормализации заменяется на "е".

```

# -*- coding: utf-8 -*-
import re
from typing import List, Tuple

RUS_ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя" # 32 буквы, без "ё"
ALPHABET_LEN = len(RUS_ALPHABET)

def normalize_text(text: str) -> str:
    """
    Оставляет только русские буквы из RUS_ALPHABET.
    Переводит в нижний регистр, заменяет 'ё' на 'е'.
    """
    text = text.lower().replace("ё", "е")
    only_letters = re.findall(r"[a-я]", text)
    only_letters = [ch for ch in only_letters if ch in RUS_ALPHABET]
    return "".join(only_letters)

def chunk(s: str, n: int) -> List[str]:
    """Разбивает строку s на куски длины n."""
    return [s[i:i+n] for i in range(0, len(s), n)]

def pad(text: str, block_size: int, filler: str = "x") -> str:
    """Дополняет текст до кратности block_size символом filler."""
    if not filler or len(filler) != 1:
        raise ValueError("filler must be a single character")
    need = (-len(text)) % block_size
    return text + filler * need

```

1) Столбцовая перестановка (маршрут по столбцам)

Схема:

1. Берем ключевое слово, например "пароль".
2. Удаляем из открытого текста все не-русские символы, пробелы и т.п.
3. Заполняем таблицу построчно, число столбцов равно длине ключа.
4. Нумеруем столбцы по алфавитному порядку букв ключа (если буквы повторяются, порядок стабилизируем индексом).
5. Криптограмма получается при выписывании столбцов сверху вниз в порядке возрастания номеров.

```

from dataclasses import dataclass
import math

@dataclass
class ColumnarTransposition:
    key: str
    filler: str = "x"

    def __post_init__(self):
        if not self.key:
            raise ValueError("Key must be non-empty")
        k = normalize_text(self.key)
        if not k:
            raise ValueError("Key must contain Russian letters")
        self.key = k

    def _order(self) -> List[int]:
        pairs = [(ch, i) for i, ch in enumerate(self.key)]
        pairs_sorted = sorted(pairs, key=lambda x: (x[0], x[1]))
        order = [i for _, i in pairs_sorted]
        return order

    def encrypt(self, plaintext: str) -> str:
        p = normalize_text(plaintext)
        n_cols = len(self.key)
        p = pad(p, n_cols, self.filler)
        n_rows = len(p) // n_cols
        table = [list(p[r*n_cols:(r+1)*n_cols]) for r in range(n_rows)]
        order = self._order()
        cipher_cols = []
        for col_idx in order:
            col = [table[r][col_idx] for r in range(n_rows)]
            cipher_cols.append("".join(col))
        return "".join(cipher_cols)

    def decrypt(self, ciphertext: str) -> str:
        c = normalize_text(ciphertext)
        n_cols = len(self.key)
        n_rows = math.ceil(len(c) / n_cols)
        if len(c) != n_cols * n_rows:
            raise ValueError("Ciphertext length must be divisible by number of columns")
        order = self._order()
        cols = chunk(c, n_rows)
        table = [[""] * n_cols for _ in range(n_rows)]
        for src, col_idx in enumerate(order):
            col = cols[src]

```

```

        for r in range(n_rows):
            table[r][col_idx] = col[r]
    plain = "".join("".join(row) for row in table)
    return plain

# Демонстрация
ct = ColumnarTransposition(key="пароль", filler="я")
text_ct = "Нельзя недооценивать противника"
c1 = ct.encrypt(text_ct)
p1 = ct.decrypt(c1)
print("Ключ:", ct.key)
print("Открытый текст:", normalize_text(text_ct))
print("Шифртекст:", c1)
print("Дешифр:", p1)

```

```

Ключ: пароль
Открытый текст: нельзя недооценивать противника
Шифртекст: еенпнзоатаьовокннеьвдирияцтия
Дешифр: нельзя недооценивать противника

```

2) Решетка Флейснера

Конструкция решетки размера $2k \times 2k$:

1. Строим базовый квадрат $k \times k$, заполненный числами $1..k^2$ по строкам.
2. Делаем повороты на 90 градусов по часовой стрелке и составляем большой квадрат:

[base, rot90] [rot270, rot180]

3. В качестве решетки берем отверстия $k \times k$ в левом верхнем углу. При 4 поворотах эти отверстия покрывают весь квадрат.

После заполнения таблицы можно либо читать построчно, либо как в методичке - по столбцам в порядке, заданном ключом длины k^2 из уникальных букв.

```

from typing import Optional

def rotate_coords(i: int, j: int, n: int, times: int = 1):
    r_i, r_j = i, j
    for _ in range(times % 4):
        r_i, r_j = r_j, n - 1 - r_i
    return r_i, r_j

class FleissnerGrille:
    def __init__(self, k: int, fill_char: str = "x"):
        if k <= 1:
            raise ValueError("k must be > 1")
        self.k = k
        self.n = 2 * k
        if not fill_char or len(fill_char) != 1:
            raise ValueError("fill_char must be a single character")
        self.fill_char = fill_char
        self.holes = [(i, j) for i in range(k) for j in range(k)]

    def _perm_from_key(self, key: Optional[str]):
        if key is None:
            return None
        k_norm = normalize_text(key)
        if len(k_norm) != self.k * self.k:
            raise ValueError("Column key length must be exactly k^2 and use Russian letters")
        if len(set(k_norm)) != len(k_norm):
            raise ValueError("Column key must contain unique letters")
        pairs = [(ch, i) for i, ch in enumerate(k_norm)]
        pairs_sorted = sorted(pairs, key=lambda x: (x[0], x[1]))
        return [i for _, i in pairs_sorted]

    def encrypt(self, plaintext: str, col_key: Optional[str] = None) -> str:
        p = normalize_text(plaintext)
        block = self.n * self.n
        p = pad(p, block, self.fill_char)
        out = []
        for off in range(0, len(p), block):
            part = p[off:off+block]
            table = [[None] * self.n for _ in range(self.n)]
            it = 0
            for t in range(4):
                for (i, j) in self.holes:
                    ri, rj = rotate_coords(i, j, self.n, t)
                    table[ri][rj] = part[it]
                    it += 1
            if col_key is None:

```

```

        out.append("".join("".join(row) for row in table))
    else:
        perm = self._perm_from_key(col_key)
        cols = ["".join(table[r][c] for r in range(self.n)) for c in range(self.n)]
        out.append("".join(cols[idx] for idx in perm))
    return "".join(out)

def decrypt(self, ciphertext: str, col_key: Optional[str] = None) -> str:
    c = normalize_text(ciphertext)
    block = self.n * self.n
    if len(c) % block != 0:
        raise ValueError("Ciphertext length must be a multiple of (2k)^2")
    out = []
    for off in range(0, len(c), block):
        part = c[off:off+block]
        table = ["" * self.n for _ in range(self.n)]
        if col_key is None:
            it = 0
            for r in range(self.n):
                for col in range(self.n):
                    table[r][col] = part[it]
                    it += 1
        else:
            perm = self._perm_from_key(col_key)
            cols = chunk(part, self.n)
            inv = [0] * len(perm)
            for src, dst in enumerate(perm):
                inv[dst] = src
            for col in range(self.n):
                col_data = cols[inv[col]]
                for r in range(self.n):
                    table[r][col] = col_data[r]

        it = 0
        res = ["" * block]
        for t in range(4):
            for (i, j) in self.holes:
                ri, rj = rotate_coords(i, j, self.n, t)
                res[it] = table[ri][rj]
                it += 1
            out.append("".join(res))
    return "".join(out)

# Демонстрация
fg = FleissnerGrille(k=2, fill_char="я")
text_fg = "Договор подписали"
c2_plain = fg.encrypt(text_fg)

```

```
c2_with_key = fg.encrypt(text_fg, col_key="шифр")
print("Текст:", normalize_text(text_fg))
print("Шифртекст без ключа столбцов:", c2_plain)
print("Шифртекст с ключом столбцов 'шифр':", c2_with_key)
print("Дешифр (без ключа столбцов):", fg.decrypt(c2_plain))
print("Дешифр (с ключом столбцов 'шифр'):", fg.decrypt(c2_with_key, col_key="шифр"))
```

```
Текст: договорподписали
Шифртекст без ключа столбцов: дорвгопоаиипслдо
Шифртекст с ключом столбцов 'шифр': ооилвопорпиддгас
Дешифр (без ключа столбцов): договорподписали
Дешифр (с ключом столбцов 'шифр'): договорподписали
```

3) Шифр Виженера

- Алфавит из 32 букв: "абвгдежзийклмнопрстуфхцчшщъыьэюя".
- "ё" приводим к "е".
- Ключ повторяем над текстом и сдвигаем каждую букву по алфавиту.

Формулы:

- $enc = (p + k) \bmod 32$
- $dec = (c - k) \bmod 32$

```

class VigenereRU:
    def __init__(self, key: str):
        k = normalize_text(key)
        if not k:
            raise ValueError("Key must contain Russian letters")
        self.key = k
        self._pos = {ch: i for i, ch in enumerate(RUS_ALPHABET)}

    def _rep_key(self, n: int) -> str:
        q, r = divmod(n, len(self.key))
        return self.key * q + self.key[:r]

    def encrypt(self, plaintext: str) -> str:
        p = normalize_text(plaintext)
        krep = self._rep_key(len(p))
        out = []
        for ch, kch in zip(p, krep):
            i = self._pos[ch]
            j = self._pos[kch]
            out.append(RUS_ALPHABET[(i + j) % len(RUS_ALPHABET)])
        return "".join(out)

    def decrypt(self, ciphertext: str) -> str:
        c = normalize_text(ciphertext)
        krep = self._rep_key(len(c))
        out = []
        for ch, kch in zip(c, krep):
            i = self._pos[ch]
            j = self._pos[kch]
            out.append(RUS_ALPHABET[(i - j) % len(RUS_ALPHABET)])
        return "".join(out)

# Демонстрация
vr = VigenereRU(key="математика")
text_vr = "криптография серьезная наука"
c3 = vr.encrypt(text_vr)
p3 = vr.decrypt(c3)
print("Ключ:", vr.key)
print("Открытый текст:", normalize_text(text_vr))
print("Шифртекст:", c3)
print("Дешифр:", p3)

```


Ключ: математика

Открытый текст: криптографиясерьезнаянаука

Шифртекст: црѣюохшкфягкьчпчалнтщца

Дешифр: криптографиясерьезнаянаука

Удобные обертки для проверки

```

def demo_columnar(text: str, key: str, filler: str = "x"):
    print("== Столбцовая перестановка ==")
    c = ColumnarTransposition(key=key, filler=filler)
    p_norm = normalize_text(text)
    print("Ключ:", c.key)
    print("Исходный:", p_norm)
    enc = c.encrypt(text)
    dec = c.decrypt(enc)
    n_cols = len(c.key)
    print("Таблица по строкам (для наглядности):")
    rows = chunk(pad(p_norm, n_cols, filler), n_cols)
    for r in rows:
        print(" ".join(list(r)))
    print("Шифртекст:", enc)
    print("Обратное преобразование:", dec)

def demo_fleissner(text: str, k: int, filler: str = "x", col_key: str = None):
    print("== Решетка Флейснера ==")
    g = FleissnerGrille(k=k, fill_char=filler)
    p_norm = normalize_text(text)
    print("Размер:", 2*k, "x", 2*k)
    print("Исходный:", p_norm)
    if col_key:
        print("Ключ столбцов:", normalize_text(col_key))
    enc = g.encrypt(text, col_key=col_key)
    dec = g.decrypt(enc, col_key=col_key)
    print("Шифртекст:", enc)
    print("Обратное преобразование:", dec)

def demo_vigenere(text: str, key: str):
    print("== Вижнер ==")
    v = VigenereRU(key=key)
    p_norm = normalize_text(text)
    print("Ключ:", v.key)
    print("Исходный:", p_norm)
    enc = v.encrypt(text)
    dec = v.decrypt(enc)
    print("Шифртекст:", enc)
    print("Обратное преобразование:", dec)

print()
demo_columnar("Пример для столбцовой перестановки", key="ключ")
print()
demo_fleissner("Пример для решетки", k=3, col_key=None)
print()

```

```
demo_vigenere("простейшая проверка", key="пример")
```

```

== Столбцовая перестановка ==

Ключ: ключ

Исходный: примердлястолбцовойперестановки

Таблица по строкам (для наглядности):

п р и м
е р д л
я с т о
л б ц о
в о й п
е р е с
т а н о
в к и х

Шифртекст: п ея л в е т в р р с б о р а к м л о о п с о х и д т ц ѝ е н и

Обратное преобразование: примердлястолбцовойперестановки


== Решетка Флейснера ==

Размер: 6 x 6

Исходный: примердлярешетки

Шифртекст: прииермерхтедлйхкшxxxxxxxxxxxxxxxxxxxxx

Обратное преобразование: примердлярешеткиxxxxxxxxxxxxxxxxxxxxx


== Виженер ==

Ключ: пример

Исходный: простейшаяпроверка

Шифртекст: юацэчхшиилфаэтьнпр

Обратное преобразование: простейшаяпроверка

```

Обратное преобразование: простейшая проверка

Как использовать

- ```
c = ColumnarTransposition(key="пароль", filler="я")
cipher = c.encrypt("ваш текст")
back = c.decrypt(cipher)
```

- ```
g = FleissnerGrille(k=3, fill_char="я")
cipher = g.encrypt("ваш текст", col_key=None)
back = g.decrypt(cipher, col_key=None)
```

- Вижнер:

```
v = VigenereRU(key="математика")  
cipher = v.encrypt("ваш текст")  
back = v.decrypt(cipher)
```

Функции принимают и возвращают только русские буквы, пробелы и знаки препинания отбрасываются.